

Opening remarks

What the application does

Course discovery
Evaluations and attribution
Profiles, memories, and history
Schedule audits
Schedules, calendar, and chat

How the application works

Logical Layers
Components and integrations
Deployment view
Chat request and SSE
Unified search pipeline
Mutations and clarification
Parallel Audit and Quality Gate
Output validation (civility / policy)
Agent Overview
Automated tests, evaluation, and observability

Closing remarks

Limitations
Future work and reflection
Questions

Atlas

AI-Assisted Schedule Builder for JHU Undergraduates

Alina Pan Jennifer He James Guo Junzhe Shi
Rachael Pei

Johns Hopkins University
Final presentation

May 5, 2026



Opening: Problem, Users, Why AI

Problem

JHU students need to discover courses, respect constraints, and build feasible schedules using SIS data and evaluations—without tab-hopping and guesswork.

Users

Undergraduates planning a term: search, shortlist, compare workload, and iterate with advising-style help.

Why AI is essential (not bolted-on)

- ▶ **Discovery:** Semantic search + match explanations over embeddings; an agent orchestrates tools instead of static forms.
- ▶ **Personalization:** Onboarding → structured memories; schedule-aware chat; natural-language add/drop/swap.
- ▶ **Reliability:** Clarification before risky writes; unified search; parallel audits with quality gates on outputs.



Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedule audits
- Schedules, calendar, and chat

How the application works

- Logical Layers
- Components and integrations
- Deployment view
- Chat request and SSE
- Unified search pipeline
- Mutations and clarification
- Parallel Audit and Quality Gate
- Output validation (civility / policy)
- Agent Overview
- Automated tests, evaluation, and observability

Closing remarks

- Limitations
- Future work and reflection
- Questions

Demo 1 — Course discovery

Goal

Open-ended query → merged ranked results from unified searchCourses with explicit matchType and readable matchExplanation; clarification when the request is ambiguous.

- ▶ Live: NL query → cards with matchType (exact / constraint / semantic / hybrid) and substantive explanations, including backfill when the agent forces a narrow search payload.
- ▶ Optional path: ambiguous intent → disambiguation UI with saved multi-turn state.
- ▶ When the query names a **professor** or **specific course**, parallel **RateMyProfessor** + **Reddit** lookups can augment internal evals; tool use stays scoped so generic browse queries stay lightweight.



Demo 2 — Eval summaries + attribution

Goal

On-demand eval summary grounded in metrics with an inspectable source table; course metrics default to **cross-term aggregation** unless the user or tool pins a specific term.

- ▶ Show: “Summarize evals” on a course card → narrative + numeric claims tied to evidence.
- ▶ Open **raw** / **source** modal: metric names, values, terms, instructors; each claim traceable to a row in the extended attribution view.
- ▶ If no backing eval rows: hide raw access and do not fabricate a summary.

Opening remarks

What the application does

Course discovery

Evaluations and attribution

Profiles, memories, and history

Schedule audits

Schedules, calendar, and chat

How the application works

Logical Layers

Components and integrations

Deployment view

Chat request and SSE

Unified search pipeline

Mutations and clarification

Parallel Audit and Quality Gate

Output validation (civility / policy)

Agent Overview

Automated tests, evaluation, and observability

Closing remarks

Limitations

Future work and reflection

Questions

Demo 3 — Onboarding, memories, history

Goal

Onboarding and chat produce structured memories; completed-course history and transcript intake feed recommendations, prerequisite logic, and audits.

- ▶ Show: onboarding or profile → **derived memories** / memory list with phrasing tuned for downstream agent use.
- ▶ Completed courses: add/remove from history; optional **transcript** paste or upload → review list with **matched** / **ambiguous** / **unmatched** before anything is saved.
- ▶ Keep one short path showing how history drives **prerequisite readiness** and exclusions in planning or audit surfaces.

Demo 4 — Audit (parallel checks + synthesis)

Goal

From the schedule context, run a **parallelized** audit that merges normalized checks into one report: workload, schedule conflicts, preference alignment for days and times, and prerequisite readiness, then apply an audit-only **quality gate** validate → one regenerate → fallback, with **partial results** when an individual check fails.

- ▶ Walk findings by category with severity and evidence, including **preference alignment** against stated days and times.
- ▶ Call out **prerequisites** and **conflicts** alongside workload so plan fit is grounded in concrete checks, not a separate opaque score.
- ▶ Note the async **offline evaluator** on completed audit sends for observability; be ready in Q&A to contrast audit vs. non-audit output policies if asked.

Demo 5 — Schedules, calendar, and schedule-aware chat

Goal

From the dashboard, open a schedule, use the **list vs. weekly calendar** view, optionally add **custom events** on the grid or by natural language, then use schedule-attached chat for NL add/drop/swap with stable orchestration and integration hardening.

- ▶ Toggle **list / calendar**; show blocks with course code, section, and local time range; mention **custom blocks** if enabled in your build.
- ▶ One NL mutation on the deployed app; metrics and search in schedule chat respect **allowlisted** resolved course context so the model cannot invent schedule rows.
- ▶ If time: walk a **clarification** loop for an ambiguous section or add, with state saved per conversation until answers resume the flow; optionally show section-aware add or swap messaging when sections collide.



Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedule audits
- Schedules, calendar, and chat

How the application works

- Logical Layers
- Components and integrations
- Deployment view
- Chat request and SSE
- Unified search pipeline
- Mutations and clarification
- Parallel Audit and Quality Gate
- Output validation (civility / policy)
- Agent Overview
- Automated tests, evaluation, and observability

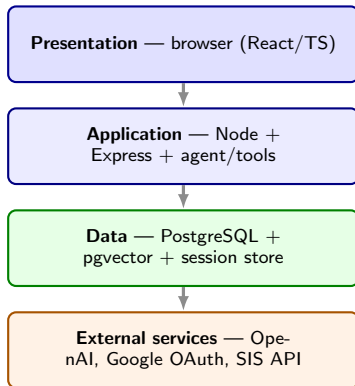
Closing remarks

- Limitations
- Future work and reflection
- Questions

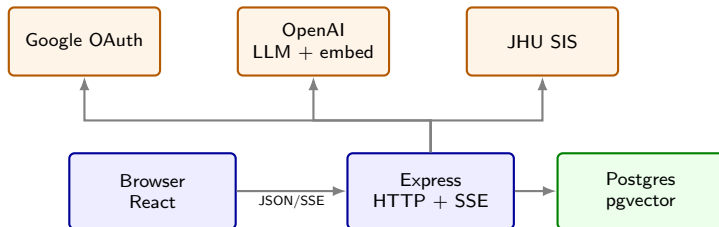
Stack: Logical Layers

Responsibilities

- ▶ **Client:** React + Vite, streaming UI, schedule views
- ▶ **API:** Express routes, auth session, agent endpoint
- ▶ **Data:** Postgres + pgvector, chat/audit persistence
- ▶ **External:** OpenAI, Google OAuth, JHU SIS



Stack: Components



Integration notes

Session cookie authenticates dashboard routes; `/api/agent` upgrades to SSE for streaming tokens and tool progress.

Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedule audits
- Schedules, calendar, and chat

How the application works

Logical Layers

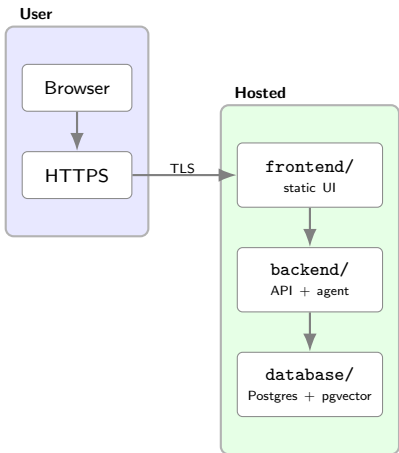
Components and integrations

- Deployment view
- Chat request and SSE
- Unified search pipeline
- Mutations and clarification
- Parallel Audit and Quality Gate
- Output validation (civility / policy)
- Agent Overview
- Automated tests, evaluation, and observability

Closing remarks

- Limitations
- Future work and reflection
- Questions

Deployment



Swap host names as needed: TLS at static frontend/; browser calls backend/; server uses database/ on managed Postgres.

Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedule audits
- Schedules, calendar, and chat

How the application works

- Logical Layers
- Components and integrations

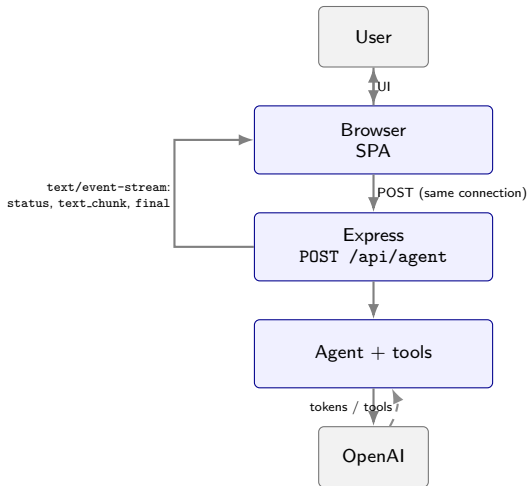
Deployment view

- Chat request and SSE
- Unified search pipeline
- Mutations and clarification
- Parallel Audit and Quality Gate
- Output validation (civility / policy)
- Agent Overview
- Automated tests, evaluation, and observability

Closing remarks

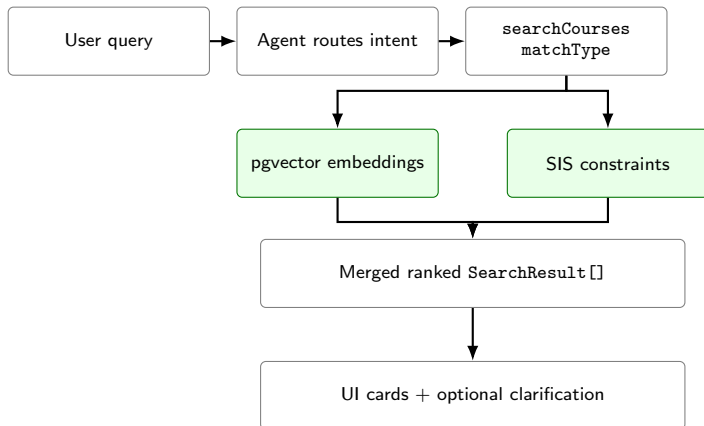
- Limitations
- Future work and reflection
- Questions

Runtime: chat request and SSE



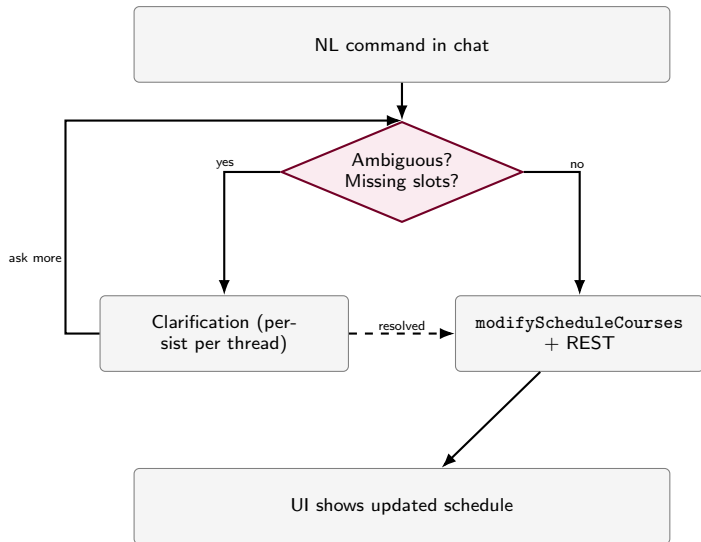
User message saved immediately; assistant text on final; partial + aborted metadata on stop.

Data flow: unified course search



One merged search surface exposes semantic, exact, and hybrid retrieval per row via `matchType`.

Control flow: NL schedule edit



Schedule mutations stay blocked until every required clarification slot is confirmed.



Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedule audits
- Schedules, calendar, and chat

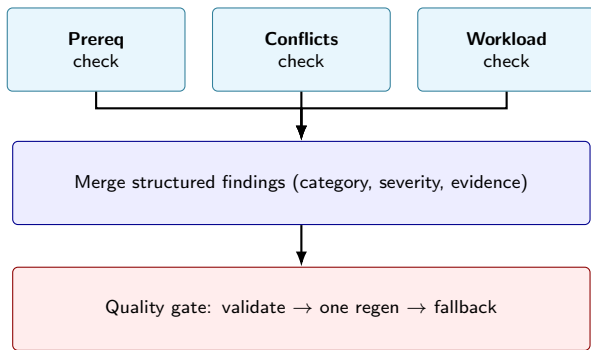
How the application works

- Logical Layers
- Components and integrations
- Deployment view
- Chat request and SSE
- Unified search pipeline
- Mutations and clarification**
- Parallel Audit and Quality Gate
- Output validation (civility / policy)
- Agent Overview
- Automated tests, evaluation, and observability

Closing remarks

- Limitations
- Future work and reflection
- Questions

Parallel Audit Synthesis and Gate



Failure isolation

One failed check still returns partial findings; gate telemetry (gate_pass, gate_fail, ...) where implemented.

Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedule audits
- Schedules, calendar, and chat

How the application works

- Logical Layers
- Components and integrations
- Deployment view
- Chat request and SSE
- Unified search pipeline
- Mutations and clarification

Parallel Audit and Quality Gate

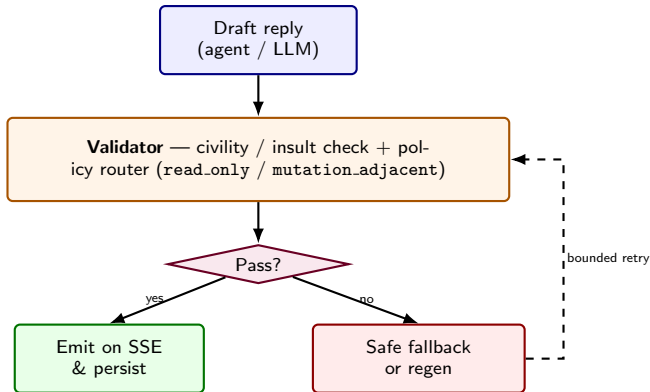
- Output validation (civility / policy)
- Agent Overview
- Automated tests, evaluation, and observability

Closing remarks

- Limitations
- Future work and reflection
- Questions

User-visible text: validator before stream

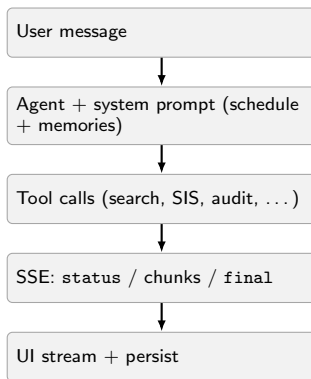
Assistant text can occasionally include **insulting** or **irrelevant** contents. A **validator** runs on draft output *before* chunks are committed to the SSE stream.



Agent Loop

Tooling

- ▶ Models: e.g. gpt-4o-mini; embeddings text-embedding-3-small
- ▶ Search, SIS details, eval summary, metrics, audits, schedule mutate
- ▶ Unified searchCourses; clarification state before writes



Opening remarks

What the application does

Course discovery
Evaluations and attribution
Profiles, memories, and history
Schedule audits
Schedules, calendar, and chat

How the application works

Logical Layers
Components and integrations
Deployment view
Chat request and SSE
Unified search pipeline
Mutations and clarification
Parallel Audit and Quality Gate
Output validation (civility / policy)

Agent Overview

Automated tests, evaluation, and observability

Closing remarks

Limitations
Future work and reflection
Questions

Testing, AI eval, observability

Non-AI (backend/ + frontend/)

- ▶ `npm run test` in `backend/` and `frontend/` (Vitest).
- ▶ `frontend/`: `npm run test:e2e` (Playwright).
- ▶ CI and unit/E2E expectations per final delivery rubric.

AI evaluation

- ▶ Golden prompts / structured checks; agent regression paths.
- ▶ Gate telemetry where the audit and output gates are wired for observability.

Coverage

- ▶ `npm run test:coverage` in `backend/` and `frontend/` for line/branch reports (Vitest + v8).
- ▶ Tie spoken numbers to what CI actually runs; note gaps honestly at Q&A.

Opening remarks

What the application does

Course discovery
Evaluations and attribution
Profiles, memories, and history
Schedule audits
Schedules, calendar, and chat

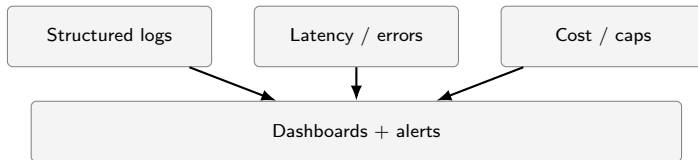
How the application works

Logical Layers
Components and integrations
Deployment view
Chat request and SSE
Unified search pipeline
Mutations and clarification
Parallel Audit and Quality Gate
Output validation (civility / policy)
Agent Overview
Automated tests, evaluation, and observability

Closing remarks

Limitations
Future work and reflection
Questions

Ops feedback loop (conceptual)



Final delivery rubric: tie spoken claims to what is actually deployed (logging, rate limits, validation). Mark items as **shipped** vs. **planned** live.

Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedule audits
- Schedules, calendar, and chat

How the application works

- Logical Layers
- Components and integrations
- Deployment view
- Chat request and SSE
- Unified search pipeline
- Mutations and clarification
- Parallel Audit and Quality Gate
- Output validation (civility / policy)
- Agent Overview
- Automated tests, evaluation, and observability**

Closing remarks

- Limitations
- Future work and reflection
- Questions

Closing: Limitations

Opening remarks

What the application does

Course discovery

Evaluations and attribution

Profiles, memories, and history

Schedule audits

Schedules, calendar, and chat

How the application works

Logical Layers

Components and integrations

Deployment view

Chat request and SSE

Unified search pipeline

Mutations and clarification

Parallel Audit and Quality Gate

Output validation (civility / policy)

Agent Overview

Automated tests, evaluation, and observability

Closing remarks

Limitations

Future work and reflection

Questions

Where Atlas still falls short

1. **Partial evaluation coverage.** Checks and QA do not yet span every intent or source; stronger semantic analysis across evals, Reddit, RMP, SIS, and chat would support deeper, more comparable scoring.
2. **Smarter agent.** We use gpt-4o-mini for cost and speed; larger or higher-capacity models (more context, stronger reasoning) would help long threads and tool use at higher token cost and latency.
3. **More edge cases.** Fine-tuned flows could cover bespoke plans: weekly templates, explicit time blocks, small constraint tweaks, and advisor-specific modes beyond generic prompts.
4. **Courses vs. sections.** Flows remain course-centric in places; richer section-level control (times, seats, cross-listing) would make adds, swaps, and audits match enrollment reality more closely.



Closing: Future work & reflection

Future work

Stronger observability dashboards; broader course history and transcript fidelity; tighter prompt and tool contracts; more automated AI evaluation on golden paths; deeper section-aware scheduling where the product roadmap allows.

Reflection

Team coordination across a long build window; using AI tools as development partners; trade-offs between latency and depth, autonomy and safety, and shipping proof over perfection.



Thank you — Questions

Questions?

Please ask questions about the project, the team, the process, the technology, the future, etc.

